



Experimentein.ai

Multimodal Similarity and Comparison of Protein
Experiments

Technical Report — Second Submission

Babyneers Team — January 2026

For more technical details check: — [Github Repo](#)



1 Project Overview

Protein-related experiments form the backbone of modern biological research, spanning expression studies, functional assays, binding experiments, and structural analyses. Despite their importance, experiments remain embedded in unstructured papers, figures, and supplementary materials rather than exposed as searchable, first-class entities. This fragmentation creates persistent challenges:

Experiments are scattered across heterogeneous document repositories with no unified indexing at the experiment level. Document-level search tools cannot pinpoint specific experiments or enable fine-grained comparison. Experiment comparison is difficult due to missing metadata, protocol variability, and inconsistent reporting. Existing similarity systems lack transparency—users cannot see *why* two experiments are similar or what evidence supports extracted parameters.

Experimentein.ai addresses these challenges by extracting experiments as structured, queryable entities from scientific papers. The system provides:

Experiment-level indexing and semantic retrieval; similarity-based exploration with acknowledged uncertainty; explicit distinction between measured, inferred, and missing parameters; and traceable provenance linking every claim back to evidence blocks in the original paper.

The platform is a research prototype for transparent discovery and comparison of experimental knowledge, not a predictive or outcome-inference system.

2 Project Architecture

The system separates extraction, storage, retrieval, and user interaction across specialized components.

2.1 Qdrant Integration for Vector Search

Qdrant is central to our semantic discovery and similarity retrieval strategy. Rather than storing full experiment records in Qdrant (which bloats vector payloads and degrades query latency), we adopt a dual-index pattern:

Qdrant stores *minimal payloads*: dense embeddings from FastEmbed paired only with unique identifiers (paper IDs, block IDs, experiment IDs). This keeps the vector index compact and search latency low. Qdrant accelerates retrieval by returning relevant identifiers in milliseconds for any semantic query. All identifier results are then resolved back to authoritative full records in Astra DB, ensuring scientific accuracy and provenance are preserved.

This separation yields several benefits: vector search remains fast even as the corpus scales; embeddings are indexed efficiently without redundant text; every result is traceable to canonical evidence in Astra; and similarity confidence scores are decoupled from payload size. The pattern ensures that researchers can explore experiments via semantic similarity without sacrificing the transparency and auditability that the system promises.

2.2 Database and Storage Strategy

We intentionally distribute responsibilities across databases to preserve auditability and maintain fast retrieval:

Astra DB (Cassandra) serves as canonical truth, storing complete structured records (papers, sections, blocks, experiments) with full JSON payloads, provenance metadata, and evidence references. This ensures reproducibility and traceability.

Qdrant acts as a retrieval accelerator, storing embeddings paired with minimal payloads (identifiers only). This keeps vector search fast and prevents bloated payloads from degrading latency. All similarity results resolve back to authoritative records in Astra.

MongoDB with Mongoose ODM handles user authentication, sessions, and credit accounting in the web application. This decouples product/account data from scientific storage and provides a flexible schema for rapidly evolving user features.

2.3 Technology Stack and Model Selection

2.3.1 PDF and Document Processing

GROBID (GitHub Open-source Reference-based PDF Information Extraction Dimensions) is a machine-learning-based PDF parser that converts scientific papers into TEI (Text Encoding Initiative) XML format. TEI XML preserves document structure, metadata, and semantic information essential for downstream processing.

lxml is a Python XML processing library used to parse TEI output into semantic blocks. **spaCy** is an industrial-strength NLP library that tokenizes, lemmatizes, and identifies named entities within extracted blocks, enabling linguistic normalization and semantic understanding of experimental text.

2.3.2 Embedding and Similarity Retrieval

FastEmbed is a lightweight, optimized embedding library that generates dense vector representations efficiently. We employ **BAAI/bge-base-en-v1.5**, a state-of-the-art English embedding model from the BAAI project that achieves competitive performance on semantic similarity tasks while maintaining reasonable dimensionality and inference latency. These embeddings are indexed in Qdrant for scalable similarity search.

2.3.3 LLM Providers and Roles

We use **OpenRouter**, a unified LLM API platform, to delegate inference to multiple third-party model providers while allowing users to select their preferred models. This approach decouples our infrastructure from any single vendor and enables cost-conscious model selection.

LFM-2.5-1.2B-Instruct is a lightweight, instruction-tuned foundation model used for section summarization. Its small size makes it cost-effective, reducing downstream token consumption by compressing sections into retrievable summaries without sacrificing semantic quality.

Llama 3 70B is a large, instruction-aligned foundation model used for experiment extraction. Its reasoning and instruction-following capabilities enable accurate parsing of complex experimental descriptions into structured fields with per-field confidence scores and inline evidence references. The large model is invoked only after relevant evidence has been retrieved, ensuring efficient token usage.

2.3.4 Web Application

Next.js 15 with **Server Actions** provides a modern React framework with server-side logic co-located with client components, simplifying data fetching, authentication, and real-time updates. Server Actions enable seamless client-server communication without explicit API route definitions.

Mongoose is an elegant MongoDB object data mapper (ODM) that provides schema validation, middleware hooks, and type safety for MongoDB operations. It handles user records, session management, and credit accounting with minimal boilerplate.

2.4 System Architecture

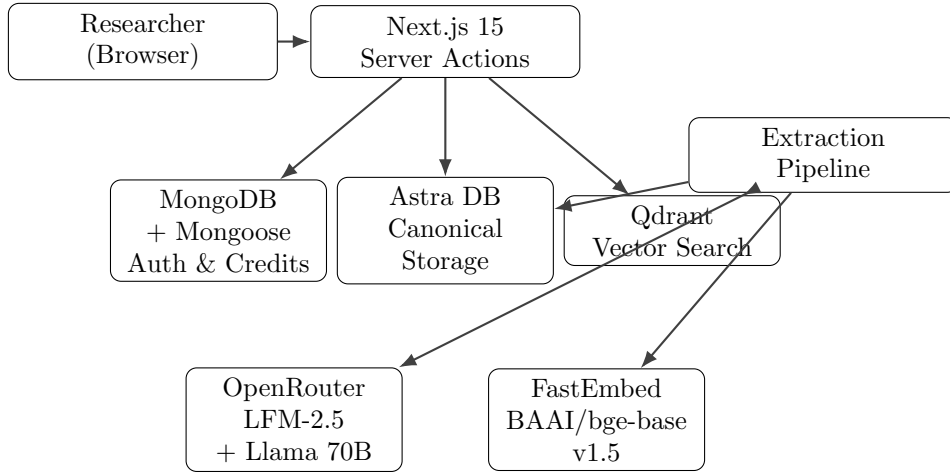


Figure 1: System architecture showing canonical storage (Astra), similarity search (Qdrant with minimal payloads), product infrastructure (MongoDB + Mongoose), and modular LLM selection via OpenRouter with embedding generation via FastEmbed.

3 Data Pipeline

The extraction pipeline transforms PDFs into structured experimental knowledge through deterministic stages that preserve uncertainty and evidence.

3.1 Pipeline Overview

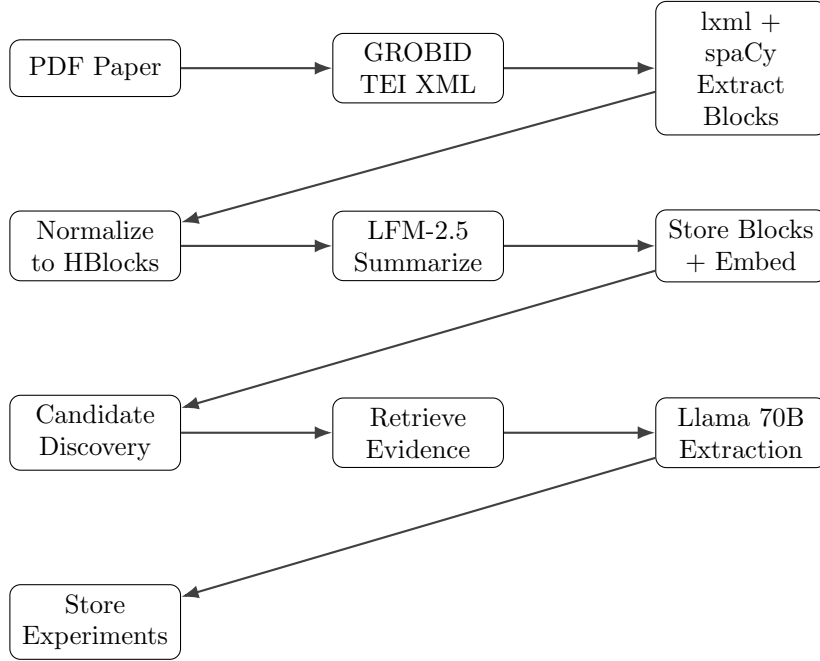


Figure 2: Deterministic pipeline from PDF ingestion to experiment extraction. Each stage preserves evidence provenance and confidence scores. LFM-2.5 handles cost-effective summarization; Llama 70B performs structured extraction with full reasoning.

3.2 Pipeline Stages

PDF to TEI XML (GROBID): Scientific PDFs are converted to structured TEI XML using GROBID, preserving document layout, hierarchy, and semantic information.

Block Extraction (lxml + spaCy): TEI documents are parsed with lxml into semantic units (titles, sections, paragraphs, tables, equations, captions). spaCy performs linguistic normalization, tokenization, and entity recognition to standardize experimental terminology.

Block Normalization: Blocks are organized into a relational schema (HBlocks) with section hierarchy, ordering metadata, and optional cross-references.

Section Summarization (LFM-2.5): Lightweight instruction-tuned summarization via OpenRouter compresses paper sections into concise retrievable summaries, guiding candidate discovery and reducing downstream token cost.

Embedding Generation (FastEmbed + BAAI/bge-base): Dense embeddings are generated for all blocks and sections using the BAAI/bge-base-en-v1.5 model, enabling semantic similarity retrieval.

Experiment Extraction (Llama 70B): The large foundation model extracts structured experiment objects from retrieved evidence blocks via OpenRouter, producing per-field confidence scores and inline evidence references. Users may select alternative models through OpenRouter’s interface.

Storage and Indexing: Complete records are persisted in Astra DB with full metadata and provenance. Embeddings and minimal identifiers are indexed in Qdrant for semantic similarity retrieval.

4 Project Timeline

4.1 Completed: 60%

Specification of an experiment-centric problem aligned with research needs. Full implementation of the PDF-to-experiment pipeline (GROBID for TEI extraction, lxml and spaCy for block parsing, LFM-2.5 for summarization, Llama 70B for structured extraction). Integration of Astra DB for canonical storage, Qdrant for semantic search, and FastEmbed with BAAI/bge-base for efficient embedding generation. Next.js 15 web application with Server Actions and Mongoose for authentication and credit management. OpenRouter integration allowing users to select preferred LLM providers and models.

4.2 Remaining: 40%

4.2.1 Web Interface Completion

Full implementation of search flows for papers, sections, blocks, and experiments with confidence-aware filtering. Experiment viewer with side-by-side parameter comparison. Evidence drawer with field-level traceability back to source blocks. Explainability panel showing extraction confidence and alternative interpretations.

4.2.2 Credit and Usage System

Complete implementation of credit accounting (reserve/execute/finalize/refund cycle) tied to OpenRouter token consumption. Per-user quota tracking and overage handling. Cost estimation before extraction based on model selection and document size.

4.2.3 Model Selection and Fine-tuning

User-facing model selector allowing choice between LFM-2.5 vs. alternative instruction-tuned models for summarization. Support for alternative large models (Claude 3.5, GPT-4, other Llama variants) for extraction via OpenRouter’s unified interface. Future work: domain-specific fine-tuning of embedding models and extraction models on curated protein experiment datasets.

4.2.4 Advanced Features

Guided exploration with suggested next searches based on experiment features and user history. Batch experiment extraction with progress tracking and failure recovery. Experiment provenance visualization showing extraction confidence, evidence blockage, and parameter uncertainty. Interactive comparison view with diff highlighting and statistical summaries.

5 Conclusion

Experimentein.ai demonstrates the feasibility of transforming fragmented experimental descriptions into a transparent, queryable experiment space. By combining deterministic document parsing (GROBID, lxml, spaCy), cost-effective summarization (LFM-2.5), sophisticated extraction (Llama 70B), and flexible LLM selection via OpenRouter, the system enables experiment-

level discovery while preserving uncertainty and provenance. FastEmbed with BAAI/bge-base provides efficient, high-quality embeddings for similarity retrieval. Next.js 15 with Mongoose delivers a responsive, maintainable web application. The modular architecture—separating canonical storage (Astra), similarity search (Qdrant), and product infrastructure (MongoDB)—ensures scientific rigor, scalability, and user transparency. This proof-of-concept shows how experiment-centric discovery can improve biological research without overreaching into predictive claims.